SLAM Homework 4:
Direct Visual odometry

Prof. Laurent Kneip

May 9th 2024

Instructions:

1. Deadline: **2024-5-23 23:59:59**
2. No handwritten homework is accepted.
3. Your homework should be submitted in PDF format and packed with your code, and the naming format of the file is *studentID-name-hw4*.
4. Please submit your homework through email to *daizj2022@shanghaitech.edu.cn* with the subject line "studentID-name-hw4"

Visual odometry (**VO**) can be divided into indirect methods and direct methods according to the usage of image information. In the direct method, we do not need to know the correspondence between points, but find them by minimizing the photometric error. In this assignment, we'll incorporate a simple **VO** algorithm based on dense direct methods and further explore the sparse direct methods using semi-dense approach.

Consider a spatial point P and two cameras at different points in time. The world coordinates of P are $[X, Y, Z]$, and it is imaged onto both cameras, with the non-homogeneous pixel coordinates denoted as $\boldsymbol{p}_1$ and $\boldsymbol{p}_2$. Our goal is to find the relative pose transformation from the first camera to the second camera. Taking the first camera as the reference frame, let the rotation and translation of the second camera be $\boldsymbol{R}, \boldsymbol{t}$ (Lie algebra $\boldsymbol{\xi}$). Assume they have same intrinsic $\boldsymbol{K}$, we can obtain entire projection equation:

$$\boldsymbol{p}_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1}\boldsymbol{K}\boldsymbol{P},$$

$$\boldsymbol{p}_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2}\boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}\right) = \frac{1}{Z_2}\boldsymbol{K}\left(\exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{P}\right)_{1:3}.$$

Among them, Z_1, Z_2 are the depth of P in the first and second camera coordinate system.

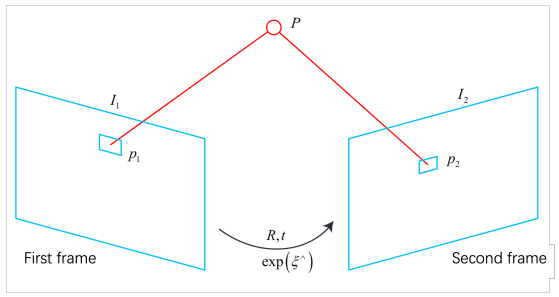

Figure 1: The direct method.

The idea of the direct method is to find the position of $\boldsymbol{p}_2$ based on the current camera pose estimate. But if the camera pose is not good enough, the appearance of $\boldsymbol{p}_2$ will be significantly different from that of $\boldsymbol{p}_1$. Therefore, in order to reduce this difference, we optimize the camera pose to find $\boldsymbol{p}_1$ that is more similar to $\boldsymbol{p}_2$. What is minimized here is the photometric error. To simplify the problem, we can use the depth channel of RGBD data. Then the pose estimation problem of the entire camera becomes ($\boldsymbol{I}_1, \boldsymbol{I}_2$ is image frame):

$$\min_{\boldsymbol{\xi}} J(\boldsymbol{\xi}) = \sum_{i=1}^{N} e_i^T e_i, \quad e_i = \boldsymbol{I}_1(\boldsymbol{p}_{1,i}) - \boldsymbol{I}_2(\boldsymbol{p}_{2,i}).$$

In order to obtain the relationship between residuals e and pose, we need to analyze their derivative relationship.

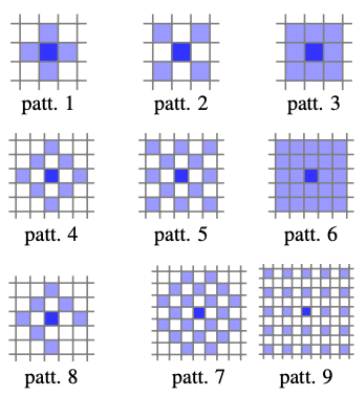

Figure 2: The Residual pattern.

There are three tasks for you:

- Analyze their derivative relationship, and write down the Jacobian matrix of error function on Lie algebra. (10%)

- Implement direct dense visual odometry between consecutive frames. The optimization can be done via Gaussian-Newton or Levenberg-Marquardt algorithm using third-party libraries. And the method may need to apply pyramidal strategy (see 2.3.1 part in DTAM[2]) to converge. Since the image size is large, running the algorithm on the entire image is time-consuming, you can resize images. Moreover, you can robustify your method by disregarding pixels whose photometric error falls above some threshold. (50%)

- Implement direct sparse visual odometry. The main idea is solve for points that have high gradient, you can use OpenCV library to get semi-dense region. In the work of DSO [1], they use different pattern to compute residuals as fig 2, please choose any two residual pattern in your implementation and compare their performance(40%).

Please use evo library to compare your trajectory with ground truth and running time of your algorithm, also describe the strategy you have used to improve algorithm and robustify your method.

Data Description:

The data comes from TUM RGB-D dataset, you can check the usage of the dataset on their official website, camera intrinsic can also be found on it.

- **rgb.txt** and **depth.txt** contains the files name and timestamp.

- **rgb/** contains the color images stored as 640x480 8-bit RGB images in PNG format.

- **depth/** contains the depth maps stored as 640x480 16-bit monochrome images in PNG format. And the depth images are scaled by a factor of 5000. Moreover, the pixels in the color and depth images correspond already 1:1.

- **groundtruth.txt** is a text file containing the translation and orientation of the camera in a fixed coordinate frame, the format of each line is **'timestamp tx ty tz qx qy qz qw'**.

Tips: Please note that before using the data, the data needs to be temporally aligned according to the collection time in order to let the rgb and depth images are paired. In principle, we can regard data whose collection time is close to a threshold as a pair of images. And the pose at the closest time is regarded as the real collection position of the image.

Notes:

1) The data description is written in data.md.

2) Both C++ and Python are acceptable. You can use the off-the-shelf libraries (e.g. ceres, g2o) to help you doing the optimize part, and OpenCV to get semi-dense region.

3) Attach your implementation with pdf in the zip. In the package, you also need to include a file named README.txt/md to identify the function of each file.

Make sure that your codes can run and are consistent with your homework. We would then arrange a meeting after the deadline in which we would ask each one of you to come in for 10 minutes to demonstrate your solution on your own computer.

4) If submitted after the deadline but still within 24hrs, a 50% penalty is applied. If submitted more than 24hrs after the deadline, a zero score will be given. In special case, please contact Prof Kneip.

References

[1] Jakob Engel, Vladlen Koltun, and Daniel Cremers. Direct sparse odometry, 2016. 3

[2] Richard A. Newcombe, Steven J. Lovegrove, and Andrew J. Davison. Dtam: Dense tracking and mapping in real-time. In *2011 International Conference on Computer Vision*, pages 2320–2327, 2011. 3